

JavaScript-09

OOP och funktionell JS

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg

ORIE
RAMM

JS



För en kanin

OOP och funktionell JS

I förra kapitlet lärde vi oss grunderna i hur man bygger klasser och objekt i JavaScript. Det ser väldigt mycket ut som Java eller C#. Men i det här kapitlet ska vi lyfta på motorhuven. JavaScript fungerar nämligen helt annorlunda i bakgrunden. När du förstår vad som döljer sig under ytan kommer du att kunna skriva mycket mer kraftfull kod, och du kommer att förstå varför vi bygger applikationer som vi gör i moderna ramverk som React.

Prototyper – Sanningen om JavaScripts "Klasser"

När vi skriver `class Bil { ... }` i JavaScript så ljugar vi egentligen lite för oss själva. Det finns inga riktiga klasser i JavaScript. Klass-nyckelordet är bara "syntaktiskt socker" – ett snyggare sätt att skriva kod på. Under huven använder JavaScript något som kallas Prototyper. Varje objekt du skapar har en osynlig länk till ett annat objekt, sin prototyp.

Om du letar efter en metod på ett objekt, och den inte finns där, letar JavaScript automatiskt vidare i prototyp-kedjan. Det är på samma sätt som om man letar efter en sak i ett hus: du börjar i rummet där du står. Hittar du den inte där, går du upp på nästa våning och letar vidare. Här är exakt vad som händer när du skriver koden `fido.andas()`.

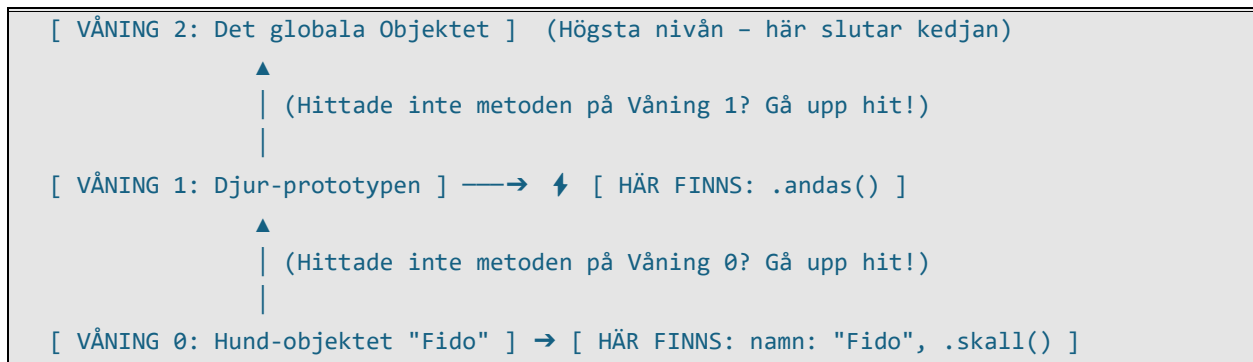


Bild 1 JavaScript letar uppåt i en kedja av objekt tills det hittar metoden.

Vi kan se ett objekts prototyp genom att kika på egenskapen `__proto__`.

```
const fordonsMall = {
  tuta() {
    console.log("Tuuut!");
  }
};

const minBil = {
  varumärke: "Volvo"
};

// Vi länkar ihop minBil med fordonsMall för hand
// (När du använder klasser och "extends" sköter JavaScript detta automatiskt)
minBil.__proto__ = fordonsMall;

minBil.tuta(); // Utskrift: Tuuut!
```

Bild 2 JavaScript hittade inte .tuta() direkt inuti minBil.

Closures (Stängningar) – Funktioner med minne

Nu lämnar vi OOP för en stund och kliver in i den funktionella programmeringens värld. Ett av de absolut mäktigaste koncepten här kallas för ett Closure. En Closure uppstår när en funktion, som ligger inuti en annan funktion, kommer ihåg variablerna från sin "förälder" – även efter att föräldrafunktionen har kört klart och stängts ner.

Tänk på det som att den inre funktionen tar med sig en liten ryggsäck med de variabler som fanns i rummet där den föddes.

Det här är perfekt om man vill skapa variabler som är helt låsta och "privata", så att ingen kan gå in utifrån och förstöra eller ändra på dem av misstag (ett alternativ till # i klasser).

```
function skapaSpelare(namn) {
  let liv = 3; // Den här variabeln är helt "gömd" inuti funktionen

  return {
    visaStatus() {
      console.log(`${namn} har ${liv} liv kvar.`);
    },
    taSkada() {
      liv--;
      console.log(`${namn} tog skada!`);
    }
  };
}

const hjalte = skapaSpelare("Link");

hjalte.taSkada();    // Utskrift: Link tog skada!
hjalte.visaStatus(); // Utskrift: Link har 2 liv kvar.

// Det går INTE att skriva hjalte.liv = 100 utifrån.
// Variabeln "liv" lever säkert i funktionens Closure-ryggsäck!
```

Bild 3 Funktionerna som returneras med ett "Closure".

Higher-Order Functions

Higher-Order functions är egentligen funktioner som hanterar funktioner. I JavaScript säger man att funktioner är **First-Class Citizens**. Det betyder att de kan behandlas precis som vilken datatyp som helst. Du kan spara en funktion i en variabel, skicka med en funktion som ett argument (en så kallad callback), eller låta en funktion skapa och returnera en helt ny funktion.

En funktion som tar emot eller returnerar en annan funktion kallas för en Higher-Order Function. Du använde dessa hela tiden i kapitel 06! `.map()`, `.filter()` och `.reduce()` är klockrena exempel på Higher-Order Functions eftersom de kräver att du skickar in en annan funktion för att de ska fungera. Här skapar vi en egen Higher-Order Function som tar emot en text och en valfri "omvandlings-funktion".

```
// En Higher-Order Function eftersom den tar emot en funktion (stilFunktion)
function formateraMeddelande(text, stilFunktion) {
  return stilFunktion(text);
}

const skrik = (t) => t.toUpperCase() + "!!!";
const viska = (t) => "shhh... " + t.toLowerCase();

console.log(formateraMeddelande("Varning", skrik)); // Utskrift: WARNING!!!
console.log(formateraMeddelande("Hemlighet", viska)); // Utskrift: shhh... hemlighet
```

Bild 4: Skicka med olika funktioner som argument.

Det luriga nyckelordet this

Nyckelordet `this` är känt för att orsaka mycket förvirring. I språk som Java betyder `this` nästan alltid "det här nuvarande objektet i klassen". Men i JavaScript beror värdet på `this` helt och hållet på hur och var funktionen anropas.

- Om du anropar en metod *inuti* ett objekt pekar `this` på objektet.
- Om du anropar en vanlig fristående funktion pekar `this` på det globala fönstret (window i webbläsaren).

Viktigt undantag: Pilfunktioner (Arrow functions `=>`) har inget eget `this`. De lånar istället `this` från koden som ligger precis runt omkring dem.

På bild 5 nedan försöker vi använda en vanlig funktion *inuti* en `setTimeout` (som styrs av webbläsarens Web-API).

```
const användare = {
  namn: "Anna",

  halsa() {
    setTimeout(function() {
      // Det här blir fel/undefined! Eftersom det är en vanlig funktion inuti
      // setTimeout har "this" tappat bort objektet Anna.
      console.log(`Hej, jag heter ${this.namn}`);
    }, 1000);
  }
};

användare.halsa(); // Utskrift efter 1 sek: Hej, jag heter undefined
```

Bild 5 Det vanliga sättet att skriva en funktion tappar bort (`this`) när den körs via asynkront.

Det absolut enklaste sättet att lösa detta i modern JavaScript är att byta till en pilfunktion. Eftersom pilfunktioner saknar ett eget `this` så lånar den istället `this` från `halsa()`-metoden, där `this` faktiskt är Anna.

```
const anvadareMedArrow = {
  namn: "Anna",
  halsa() {
    setTimeout(() => {
      // Pilfunktionen sparar och lånar "this" från omgivningen!
      console.log(`Hej, jag heter ${this.namn}`);
    }, 1000);
  }
};

anvadareMedArrow.halsa(); // Utskrift efter 1 sek: Hej, jag heter Anna
```

Bild 6 Genom att använda en arrow function (`=>`) behåller JavaScript rätt referens.

Minnet

För att förstå varför JavaScript beter sig som det gör – och hur avancerade koncept som *Closures* och *Prototyper* faktiskt fungerar – måste vi titta på hur JavaScript-motorn hanterar minnet under huven. När du förstår detta kommer du att kunna skriva kod som körs snabbare, drar mindre minne och undviker svårfångade buggar.

Stacken och Heapen (Var sparas din data?)

JavaScript-motorn (t.ex. V8 i Google Chrome eller Node.js) använder två helt olika platser i datorns minne för att spara variabler och objekt

- **Call Stack (Stacken) – Snabb och strikt**

Stacken är en supersnabb, organiserad minnesstruktur som fungerar enligt principen "sist in, först ut". Här sparas alla **primitiva datatyper** direkt (strängar, nummer, booleans, undefined, null). Här hålls också koll på **vilken funktion som körs just nu** (Execution Context). Så fort en funktion har kört klart poppar den ut ur stacken och dess lokala variabler rensas bort direkt.

- **Memory Heap (Heapen) – Stor och flexibel**

Heapen är ett stort, ostrukturerat minnesutrymme för saker som kräver mycket plats och kan ändra storlek dynamiskt. Här sparas alla **komplexa datatyper** (Objekt, Arrayer

och Funktioner). Eftersom objekt kan växa, kan de inte tryckas in i den strikta stacken. Istället läggs själva objektet i heapen, och i stacken sparas bara en **minnesadress (referens)** som pekar på var i heapen objektet ligger.

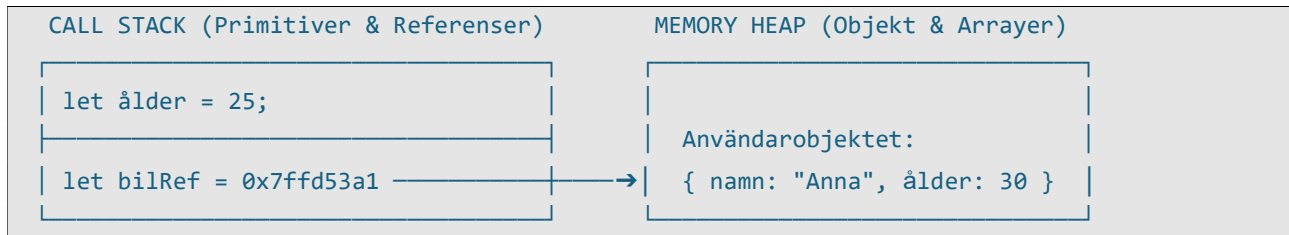


Bild 1 Stacken sparar värdet på enkla variabler direkt, men för objekt sparar den bara en referens till heapen.

Det stora problemet: Kopiera via referens

Det är den här minnesuppdelningen som gör att objekt beter sig annorlunda än vanliga variabler när man kopierar dem. Du kopierar nämligen bara *vägvisaren*, inte själva saken.

```
javascript  
let nummer1 = 10;  
let nummer2 = nummer1; // Värdet 10 kopieras direkt in i stacken  
nummer2 = 20;  
console.log(nummer1); // Utskrift: 10 (Helt oförändrad)  
  
// --- MED OBJEKT HÄNDER NÅGOT ANNAT: ---  
const bil1 = { modell: "Volvo" };  
const bil2 = bil1; // Du kopierar bara minnesadressen, inte bilen!  
  
bil2.modell = "Tesla"; // Du ändrar i heapen via bil2:s vägvisare  
  
console.log(bil1.modell); // Utskrift: "Tesla" (Hoppsan! bil1 ändrades också!)
```

Bild 2 Eftersom både bil1 och bil2 pekar på exakt samma ställe i Memory Heap, kommer en ändring i det ena objektet att påverka det andra.

Closures under huven (Minnesryggsäcken)

I kapitel 06 lärde vi oss att en **Closure** innebär att en inre funktion kommer ihåg variabler från sin föräldrafunktion, även efter att föräldrafunktionen har kört klart. Nu kan vi förklara *varför* det är möjligt tack vare heapen.

Normalt sett rensas en funktions variabler ut ur stacken så fort funktionen har kört klart (return). Men, om en inre funktion refererar till en variabel i föräldrafunktionen, flyttar JavaScript-motorn automatiskt den variabeln från den snabba stacken till **Heapen**. Eftersom objekt i heapen lever kvar så länge någon pekar på dem, kommer den inre funktionen ihåg variabeln för alltid. Det är detta som är funktionens "minnesryggsäck".

1. Funktion A startar → Skapar variabeln 'liv = 3'.
2. Funktion A stängs ner → Tas bort från Call Stack.
3. Variabeln 'liv' överlever! → Ligger kvar i Memory Heap eftersom Funktion B (den inre funktionen) fortfarande pekar på den.

Bild 3: Genom att flytta data till heapen tillåter JavaScript att funktioner har ett eget dolt, permanent minne.

Hur man undviker minnesläckor (Garbage Collection)

JavaScript har en inbyggd städare som heter **Garbage Collector**. Dess jobb är att åka runt i Memory Heapen och rensa bort objekt som inte används längre för att frigöra RAM-minne.

Städaren använder en regel som kallas *Mark-and-Sweep*. Den startar från den globala nivån (window) och kollar vilka objekt den kan nå. Alla objekt som inte går att nå via någon variabel är skräp, och rensas bort.

Hur du skapar en minnesläcka (och hur du stoppar den)

En minnesläcka sker när du råkar behålla en referens (vägvisare) till ett objekt som du egentligen inte behöver längre, vilket gör att städaren inte får kasta bort det.

Det vanligaste felet i frontend-utveckling är glömda händelselyssnare (Event Listeners) eller dolda globala variabler.

```
// EXEMPEL 1: Glömd global variabel
function kraschaMinne() {
  // Genom att missa "let" eller "const" blir variabeln global (window.data)
  // Den kommer ALDRIG att rensas bort av Garbage Collector!
  läcktData = new Array(1000000);
}

// EXEMPEL 2: Borttagna HTML-element med levande lyssnare
const knapp = document.querySelector('#minKnapp');
knapp.addEventListener('click', () => {
  console.log("Klick!");
});

// Du tar bort knappen från skärmen:
knapp.remove();

// FELET: Knappen ligger fortfarande kvar och drar RAM-minne i Heapen,
// eftersom addEventListener-funktionen fortfarande pekar på den!
```

Bild 4 Hur en minnesläcka uppstår.

När du tar bort element eller stänger ner komponenter (vilket händer hela tiden i React!), se till att städa efter dig.

```
// Ta bort lyssnaren innan du kastar elementet, så städas det bort direkt!
knapp.removeEventListener('click', minFunktion);
```

Bild 5 Borttagning av event listener.

VIKTIGT !

Att glömma kvar en event listener på ett borttaget element kan skapa minnesluckor.
